

# MagicCode Super-Resolution Software Release Notes

Document Version: v2.0.0. Release Date: September 2, 2026.

## 1. Summary

MagicCode Super-Resolution Software v2.0.0 publishes four public `alg_mode_e` values (spatial Speed/Balanced and temporal Speed/Balanced) and documents the session-API contract for temporal super-resolution on Android Vulkan/OpenGL ES, Apple Metal, and Windows Vulkan / Direct3D 11 / OpenGL 4.3.

## 2. Key Changes

- Public version string is v2.0.0 (`MC_GetVersion`).
- `alg_mode_e`: `SPATIAL_SPEED_MODE=0`, `SPATIAL_BALANCED_MODE=1`, `TEMPORAL_SPEED_MODE=2`, `TEMPORAL_BALANCED_MODE=3`.
- Temporal scaler range is (1, 8]. Spatial remains [1.0, 8.0] subject to backend and model.
- Formal temporal entry is `MC_Init` / `MC_Process` / `MC_Uninit` with `magic_frame_t` + `temporal_frame_t` (color/output, depth, motion, jitter, `frame_index/reset_history`, camera, `gpu_context`, command-buffer).
- `MC_Enable` still processes one color texture and is not a complete temporal entry.
- Windows product archives: `lib/windows/libmagic_sr.lib` and `libmagic_enable_sr.lib`. Enable does not accept `MAGIC_BACKEND_D3D11`.
- Both temporal modes default to deriving a missing reactive mask (and approximate transparency on the Metal/Vulkan/GLES Speed path) and to depth+MV history rejection. Explicit masks win per channel.

## 3. Compatibility

| Platform             | GPU backends                    | Temporal                          |
|----------------------|---------------------------------|-----------------------------------|
| Android              | Vulkan, OpenGL ES               | Temporal GPU yes; CPU temporal no |
| iOS / iPadOS / macOS | Metal                           | Temporal GPU yes; CPU temporal no |
| Windows              | Vulkan, Direct3D 11, OpenGL 4.3 | Temporal GPU yes; CPU temporal no |

- Operating systems: Windows 10 64-bit and above, Android 8.0 and above, iOS 13.0 and above, iPadOS 13.0 and above, and macOS 12.0 and above.
- Input resolution: 64×64 to 4032×4032.

- CPU SIMD (spatial CPU paths): AVX2 on x86\_64, NEON on ARM64. CPU temporal is unavailable.

## Temporal Super-Resolution

v2.0.0 adds temporal super-resolution through the session API: libmagic\_sr.a (Windows: libmagic\_sr.lib) + mc\_interface.h + MC\_Init / MC\_Process / MC\_Uninit. MC\_Enable remains a single-texture spatial helper. It may accept a temporal alg\_mode\_e value by range check, but it cannot supply per-frame depth, motion, or jitter and must not be documented as a complete temporal entry.

## Modes

| Value | Enumerator             | Role                                |
|-------|------------------------|-------------------------------------|
| 0     | SPATIAL_SPEED_MODE     | Spatial, throughput first           |
| 1     | SPATIAL_BALANCED_MODE  | Spatial, quality/cost trade-off     |
| 2     | TEMPORAL_SPEED_MODE    | Temporal, throughput first          |
| 3     | TEMPORAL_BALANCED_MODE | Temporal, canonical FSR-family path |

- Temporal scaler\_factor is (1, 8]; 1.0 itself is rejected. Spatial remains [1.0, 8.0].
- TEMPORAL\_SPEED\_MODE on Metal, Vulkan, and OpenGLS currently uses a lightweight Convert → Activate → Upscale pipeline. Direct3D 11 and desktop OpenGL 4.3 Speed use the FSR-family temporal path (same family as Balanced, not a separate customer API).
- TEMPORAL\_BALANCED\_MODE uses the canonical FSR-family path on every GPU temporal backend. Desktop Balanced edge-resolve is supported on Windows Vulkan, Windows Direct3D 11, desktop OpenGL 4.3, and Apple Metal. Android Vulkan and Android OpenGLS do not run edge-resolve. If enable\_sharpening is on, RCAS runs after edge-resolve.
- CPU backends (X86 / NEON) cannot run temporal modes.
- The API accepts (1, 8]. Exact 2× has optimized kernels. Not every scale or device is claimed as device-run.
- Internal pipeline names are implementation detail, not public configuration.

## Minimum per-frame inputs

- magic\_frame\_t.image\_in / image\_out: caller-owned color and output GPU resources.
- magic\_frame\_t.frame: non-NULL temporal\_frame\_t\* with struct\_size = sizeof(temporal\_frame\_t).
- temporal\_frame\_t.depth and motion at input resolution.
- jitter\_offset\_x/y in input pixels, top-left, +X right, +Y down.
- frame\_index / reset\_history; camera\_near / camera\_far / camera\_fov\_y (radians).
- gpu\_context at MC\_Init: Vulkan requires physical\_device + device; D3D11 requires device; Metal device may be NULL; GL/GLES use the current context (library never makes current).
- Vulkan temporal requires magic\_frame\_t.command\_buffer to be a recording VkCommandBuffer. Metal command\_buffer is optional. D3D11 / GL / GLES leave it NULL.
- Create-stage depth\_reversed / depth\_infinite / hdr\_color on input\_param\_t (all-zero = conventional-Z, finite far, LDR). Immutable for the handle lifetime.

## Motion, depth, and masks

- Motion is current → previous. The library never negates MVs.
- `motion_vector_scale_x/y` is the only MV numeric conversion. (0,0) defaults to (input\_width, input\_height). Mixed zero is rejected.
- Depth is GPU device depth in [0, 1]. Linear view-space depth is not accepted.
- reactive / transparency are optional. Explicit caller textures win per channel.
- When omitted, the library derives an approximate mask. That is not a semantic transparency mask.
- Speed on Metal/Vulkan/GLES may derive both missing reactive and an approximate transparency. FSR-family paths (Balanced on all backends; Speed on D3D11/desktop GL) derive missing reactive only and bind an internal zero transparency texture.
- Both temporal modes enable depth+MV history rejection by default.
- Diagnostic only: `MAGICSR_TAAU_AUTO_MASK=0` and `MAGICSR_TAAU_HIST_REJECT=0` can turn those defaults off. They are not a customer configuration path.

## 4. Migration Notes

- From v1.2.x: keep numeric 0/1 for spatial modes (now named `SPATIAL_*`). Add 2/3 only when integrating temporal.
- Replace any `MC_Process(handle, image_in, image_out)` usage with `MC_Process(handle, magic_frame_t*)`.
- Set `input_param_t.struct_size` and `temporal_frame_t.struct_size`.
- Do not treat Enable temporal enums as a full temporal integration.
- Review Privacy Policy and Data Reporting Notice before distributing applications that include the SDK or plugins. Those legal documents are unchanged by this technical release.

## 5. Known Limitations

- Not every backend, scale, or device class is device-run. Distinguish API support from verified configurations.
- Derived masks are approximations. Omitting transparency is not equivalent to a semantic transparency mask.
- Vulkan temporal requires a recording command buffer and explicit layouts; the library does not begin, end, or submit it.
- Performance depends on device, driver, backend, resolution, scale, and motion quality.

## 6. Validation Snapshot

API contracts are defined by `interface/mc_interface.h` and the GPU host implementations.

Compile/static coverage includes several temporal rates; that is not a device-run of every rate. Plugin Enable smoke remains spatial.